

Scope note — GACAR amendment / "Latest Changes" tracker

SECTION 06-operations-it TYPE document STATUS **active** OWNER Founder UPDATED 2026-06-16

Status: design only, no code. Scopes the "Regulatory Diff Checker" idea ("GACAR Part 61 amended 2 days ago") against Fly GACA's actual data architecture.

The ask

A tool that monitors GACA updates — a "Latest Changes" list (e.g. *"GACAR Part 61 amended 2 days ago"*) so the platform becomes the go-to for regulatory currency.

The phrase "monitors GACA updates" hides three very different problems. They must not be conflated, because their cost differs by an order of magnitude.

What the repo actually gives us today

Grounded in the data, not assumptions:

Fact	Evidence	Consequence
Part texts are in-repo	<code>assets/data/parts/part-*.html</code> — 76 files (74 Parts + index)	We can hash and diff content locally , no scraping needed.
Every index carries a build stamp	<code>gacar-index.json</code> , <code>library-search.json</code> , etc. each have a top-level <code>generated: "YYYY-MM-DD"</code> (e.g. <code>2026-05-24</code>)	A "data current as of ..." freshness line is shippable today with zero new data.
No per-document amendment/version field	union of keys in <code>library-search.json</code> entries = <code>{d,b,u,x}</code> ; <code>gacar-index.json</code> docs carry no <code>amended</code> / <code>edition</code> / <code>revision</code>	We don't know <i>when GACA last changed a Part</i> — only when we last re-ingested it.
CI is push/PR only — no scheduled job	<code>.github/workflows/ci.yml</code> on: <code>[pull_request, push]</code> ; no <code>schedule: /cron</code>	There is no existing place external monitoring could hang off.
Source PDFs are not in the repo	<code>library/PDFs/</code> is a lone <code>.gitkeep</code>	We can't diff against GACA's authoritative artifact locally.

Fact	Evidence	Consequence
A no-dep data checker already runs in CI	<code>scripts/check-data.js</code> (validates counts vs README)	Natural home to add content hashing — same shape, same runner.

Three tiers of "tracking" (do not conflate)

Tier 1 — Freshness stamp · *shippable today, ~0.5 day*

Surface the existing `generated` date on the Library and per-document pages: *"Library data current as of 24 May 2026 — always verify against the latest GACA publication."* Reinforces the core "verify against GACA" contract. No new data, no pipeline. This is pure honesty about data age, not change detection — but it's the highest value-per-effort item here.

Tier 2 — Local content-diff feed · *the recommended MVP, ~2–3 days*

Detect when **Fly GACA's own ingested copy** of a Part changes (i.e. when the maintainer re-ingests an updated reg), and publish that as a "Latest Changes" feed. This is real change detection, fully local, and the honest version of the ask — it says *"our copy of Part 61 changed on 24 May"*, never *"GACA amended Part 61"* (which we can't prove without monitoring GACA).

Data model — a committed manifest, regenerated each build:

```
// assets/data/corpus-manifest.json (committed; the diff baseline)
{
  "generated": "2026-05-24",
  "docs": [
    { "id": "part-61", "title": "Flight Crew Licensing", "kind": "regulation",
      "sha256": "9f2c...", "bytes": 184213 }
  ]
}
```

```
// assets/data/corpus-changes.json (generated; what the UI reads)
{
  "generated": "2026-05-24",
  "changes": [
    { "id": "part-61", "title": "Flight Crew Licensing",
      "status": "modified", "date": "2026-05-24" },
    { "id": "part-107", "title": "...", "status": "added", "date": "2026-05-24" }
  ]
}
```

Pipeline — extend `scripts/check-data.js` (or a sibling `build-manifest.js`):

1. SHA-256 each `parts/*.html` + each handbook/reference doc (Node `crypto`, no deps — fits the "no dependencies" rule the script already states).

2. Diff fresh hashes against the committed `corpus-manifest.json` → emit `corpus-changes.json`, stamping today's date on each `added` / `modified` / `removed`.
3. Rewrite `corpus-manifest.json` with the new hashes.
4. Run it in `ci.yml` so a content change in a PR shows up as a manifest diff in review.

UI — a "Latest Changes" card on the Library landing + an optional badge on a changed Part ("Updated 24 May"). Reads `corpus-changes.json`; degrades silently if absent (same pattern as `study-progress.js`). Each entry links to the Part and re-states "verify against GACA".

Honest limit, stated in the UI: this tracks *our re-ingestion*, not GACA's gazette. If we re-ingest weekly, "changed 2 days ago" means our copy, not necessarily a fresh GACA amendment.

Tier 3 — True GACA monitoring · *the expensive, brittle tier* — *defer*

Actually detecting that **GACA itself** amended a Part means polling `gaca.gov.sa`, fetching the current PDFs, and diffing against a stored baseline — on a schedule, unattended.

- New infra: a scheduled GitHub Action (or Cloud Function) hitting GACA, change-detecting, and opening a PR / firing a notification.
- Brittle by nature: depends on GACA's site structure and publishing cadence; breaks when they redesign. Needs the source PDFs (currently external) brought into the pipeline.
- Risk: a false "amended" claim on a regulatory platform is a credibility hit. Tier 3 needs human-in-the-loop confirmation before anything says "GACA amended X".

This is the part the original recommendation glossed over. It's a genuine project, not a UI feature.

Recommendation

Ship **Tier 1 now** (freshness stamp — trivial, on-mission). Build **Tier 2** as the MVP "Latest Changes" feed — it's real, local, low-risk, and reuses `check-data.js` + the existing `generated`-stamp convention. Treat **Tier 3** as a separate, later initiative with its own reliability budget and a human confirmation step before any "GACA amended" wording ships.

Net: the feature is feasible and worth doing — but the valuable, achievable 80% is local content-diffing of our own corpus, **not** live GACA surveillance.

Open questions for the maintainer

1. How is `parts/*.html` regenerated — a script in-repo, or an external ingestion step? (Determines whether Tier 2's hash step slots into an existing build or needs a new entry point.)
2. Desired granularity — whole-Part ("Part 61 changed") or section-level ("§61.105 changed")? Section-level needs stable per-section anchors (the search index already uses `#sec-...` URLs, so section hashing is feasible but is a Tier 2.5 increment).

3. Should "Latest Changes" also cover handbooks / the 190 reference docs, or GACAR Parts only for v1?